REKIK CAFÉ

Database Workflow & Table Usage Guide

A practical guide to how the café database tables connect, how an order moves from customer submission to payment verification, and what each table is responsible for.

Designed for: Supabase PostgreSQL and the Rekik Café web application.

Core workflow: Customer order → waiter decision → payment proof → cashier review → notifications and audit history.

Important: Tables store data; your backend must implement the workflow and enforce permissions. This guide does not imply that all security policies, Storage permissions, or notification delivery are already configured.

1. Complete Order Workflow

1. Customer places an order

Save or identify the customer, create the order, and insert one row per ordered item.

Tables: `customers` · `customer_addresses` · `orders` · `order_items`

2. Assign a waiter

Assign an eligible waiter and notify them that an order needs review.

Tables: `staff_profiles` · `orders` · `notifications`

3. Waiter approves or rejects

Record the decision and reason, update the order state, and add a status-history entry.

Tables: `order_approvals` · `orders` · `order_status_logs`

4. Customer submits payment proof

After waiter approval, upload a screenshot to private Supabase Storage and save its path and metadata.

Tables: `payments` · `payment_proofs`

5. Cashier reviews the proof

An authorized cashier approves or rejects the payment and records the decision and reason.

Tables: `payment_reviews` · `payments` · `orders`

6. Notify and preserve history

Notify the customer and assigned waiter; keep the order timeline and admin audit trail.

Tables: `notifications` · `order_status_logs` · `audit_logs`

Possible workflow outcomes

If the waiter rejects the order, record the reason and notify the customer. If the cashier rejects payment, show the reason and allow corrected proof if your business rules permit it. Uploading payment proof must not, by itself, mark a payment as verified.

2. How to Use Each Table

A. Website and menu

Table	Purpose
`about_content`	Website About page text, story, and images.
`categories`	Organize menu items into groups such as coffee, tea, meals, and desserts.
`products`	Menu item details, price, image, availability, and category relationship.
`promotions`	Discounts and special offers.
`promotion_products`	Links promotions to the products included in them.
`contact_messages`	Messages submitted through the website contact form.
`store_settings`	Café information and configurable website/store settings.

B. Customers and orders

Table	Purpose
`customers`	Customer profile information.
`customer_addresses`	Addresses associated with a customer, when delivery is supported.
`orders`	Order owner, assigned waiter, total, and current order/approval state.
`order_items`	Products, quantities, and price snapshots for each order.

C. Waiter approval and tracking

Table	Purpose
`staff_profiles`	Connect Supabase Auth accounts to roles such as admin, waiter, and cashier.
`order_approvals`	History of waiter approval/rejection decisions.
`order_status_logs`	Timeline of order status changes, including who made each change and when.
`notifications`	In-app notification records for customers and staff.

D. Payments

Table	Purpose
`payments`	Payment amount and payment status for an order.
`payment_proofs`	Private Storage file path, submission time, and proof-review status.
`payment_reviews`	Cashier decision history, including rejection reasons.

E. Administration

Table	Purpose
`audit_logs`	Important administrative and security-sensitive activity history.
`users`	Optional legacy table; use only if the application explicitly depends on it.

3. How the Tables Connect

The main relationships should follow this structure. Actual foreign-key column names must match the SQL schema installed in Supabase.

• Supabase Auth user → staff_profiles

• customers → customer_addresses

• customers → orders

• staff_profiles (assigned waiter) → orders

• orders → order_items

• categories → products → order_items

• orders → order_approvals

• orders → payments → payment_proofs → payment_reviews

• orders → order_status_logs

• orders and staff/customer recipients → notifications

• staff_profiles → audit_logs (where the audit actor is a staff member)

4. Suggested Order Statuses

Stage	Example status	Meaning
Order submitted	`pending_waiter`	Waiting for waiter review.
Waiter approved	`awaiting_payment`	Customer may submit payment proof.
Proof submitted	`payment_under_review`	Waiting for cashier verification.
Cashier approved	`payment_verified`	Payment has been accepted.
Order completed	`completed`	Order has been fulfilled.
Waiter rejected	`rejected_by_waiter`	Waiter declined the order.
Payment rejected	`payment_rejected`	Cashier rejected the proof.
Order cancelled	`cancelled`	Order cancelled under applicable rules.

These are suggested status values, not a guarantee that the schema already enforces them. Use the actual column names and types in your installed schema, and add constraints or enums if you want PostgreSQL to reject invalid states.

5. Recommended Implementation Order

1. **Build the public menu.** Read categories, products, promotions, about_content, and store_settings.
2. **Build customer ordering.** Identify the customer, create an orders row, insert order_items, and record the initial status.
3. **Build waiter assignment.** Select an eligible waiter, save the assignment, and create a notification.
4. **Build waiter approval.** Validate role and assignment on the server; record decision/reason; update the order; add a status log.
5. **Build payment submission.** Permit proof submission only after waiter approval; upload to private Storage and create payment/proof records.
6. **Build cashier verification.** Authorize the cashier, record approval or rejection and reason, and update payment/order state.

7. **Build notifications and admin history.** Notify customer and waiter; expose authorized admin views of order history, payment reviews, and audit logs.

6. Essential Security Requirements

• Enable Row Level Security (RLS) on tables exposed through the Supabase API.

• Customers should only access their own records and orders.

• Waiters should only make decisions on orders assigned to them.

• Cashiers should be authorized to review payment proofs; they should not be able to arbitrarily change waiter decisions.

• Customers must not be allowed to mark their own payment as verified.

• Store payment screenshots in a private Supabase Storage bucket with restrictive Storage policies.

• Use a transaction or secure server-side function for related changes, so an approval and its required history records succeed together.

• Create notifications only after the underlying transaction succeeds; use a server-side delivery mechanism for email or external notifications.

7. Important Implementation Notes

• **Price history:** Store the price at the time of ordering in order_items so later menu price changes do not distort old order totals.

• **Payment proof:** Uploading a screenshot is not proof that money has been received. Only an authorized cashier or trusted server-side process should mark payment as verified.

• **History tables:** order_status_logs tracks the order timeline; payment_reviews tracks cashier decisions; audit_logs tracks broader administrative/security-sensitive actions. They serve different purposes.

• **Database vs. application:** Tables store data. Backend logic, authorization policies, Storage rules, and notification delivery must be implemented and tested separately.

• **Existing projects:** If the schema is already installed, check the actual table and column definitions before using example status values or foreign-key assumptions.